

Importing the necessary libraries for the project

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
```

```
print(data.keys())
```

```
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
```

```
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
df.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.80
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20

5 rows × 31 columns

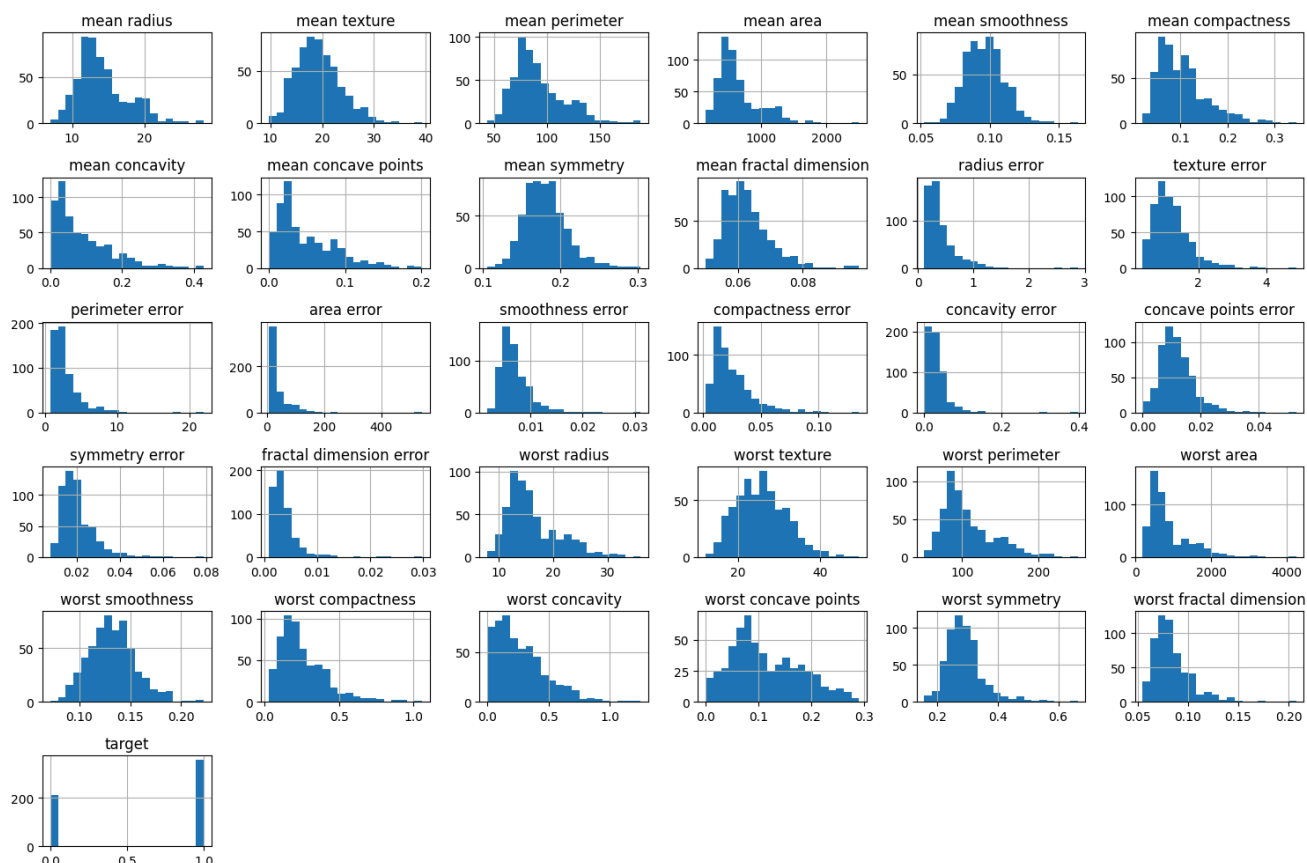
Checking for missing values

```
df.isna().sum()
```

	0
mean radius	0
mean texture	0
mean perimeter	0
mean area	0
mean smoothness	0
mean compactness	0
mean concavity	0
mean concave points	0
mean symmetry	0
mean fractal dimension	0
radius error	0
texture error	0
perimeter error	0
area error	0
smoothness error	0
compactness error	0
concavity error	0
concave points error	0
symmetry error	0
fractal dimension error	0
worst radius	0
worst texture	0
worst perimeter	0
worst area	0
worst smoothness	0
worst compactness	0
worst concavity	0
worst concave points	0
worst symmetry	0
worst fractal dimension	0
target	0

Visualising Features distribution

```
import matplotlib.pyplot as plt
import seaborn as sns
df.hist(figsize=(15,10), bins=20)
plt.tight_layout()
plt.show()
```



Splitting of dataset into training and testing

```
from sklearn.model_selection import train_test_split
X = df.drop('target', axis=1)
y = df['target']

# Split the data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y)

# Check the shape of the split datasets
print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
print("Training target:", y_train.shape)
print("Testing target:", y_test.shape)
```

Training features: (455, 30)
 Testing features: (114, 30)
 Training target: (455,)
 Testing target: (114,)

Applying standardScaler to normalize features

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Initializing and training KNeighborsClassifier with an initial k value

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

Testing different k values (1, 3, 5, 7, 9, 11) using cross-validation

```
from sklearn.model_selection import cross_val_score
k_values = [1, 3, 5, 7, 9, 11]
cv_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)

    scores = cross_val_score(
        knn,
        X_train,
        y_train,
        cv=5,
        scoring='accuracy'
    )

    cv_scores.append(scores.mean())

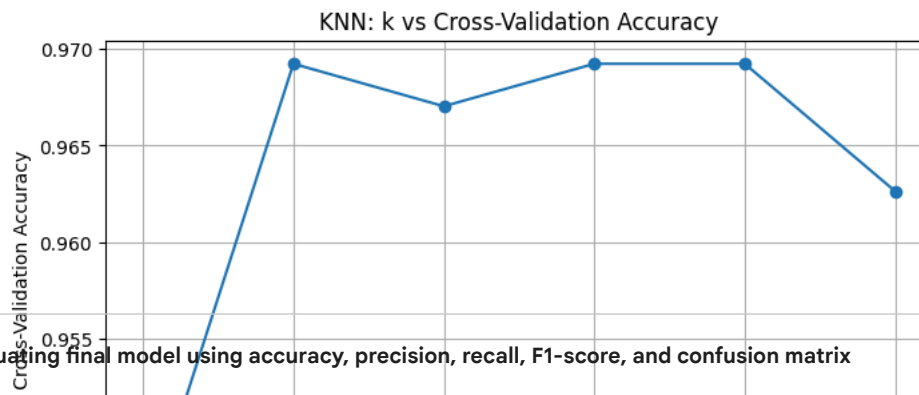
results = pd.DataFrame({
    'k': k_values,
    'Mean CV Accuracy': cv_scores
})

print(results)
```

	k	Mean CV Accuracy
0	1	0.945055
1	3	0.969231
2	5	0.967033
3	7	0.969231
4	9	0.969231
5	11	0.962637

Plotting k vs. accuracy to identify the optimal parameter

```
plt.figure(figsize=(8,5))
plt.plot(k_values, cv_scores, marker='o', linestyle='-')
plt.title('KNN: k vs Cross-Validation Accuracy')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Mean Cross-Validation Accuracy')
plt.xticks(k_values)
plt.grid(True)
plt.show()
```



```

y_pred = knn.predict(X_test)
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-score : {f1:.4f}")

```

```

Accuracy : 0.9561
Precision: 0.9589
Recall   : 0.9722
F1-score : 0.9655

```

Testing predictions on new sample data points

Using numpy to create random numbers for the features considering the minimum and maximum of each feature column

```

new_sample = pd.DataFrame(columns=X.columns)
for col in X.columns:

```